

Estudo aplicado de protocolos, c/ aplicabilidade no Projeto Integrador

Erick Jheimes Bieniek
Grazieli Julia Piekas
Samuel Martini

O presente escrito possui por objetivo realizar um estudo aplicado acerca dos protocolos que foram utilizados para a elaboração do trabalho da disciplina de Projeto Integrador V. Para a execução da atividade, em vista disso, serão analisados três dos protocolos que foram utilizados para a execução do projeto, estes que são: TCP/IP, HTTP/REST e JWT. Para cada protocolo, nesse intuito, apresenta-se a fundamentação teórica seguida de demonstrações práticas com capturas reais de tráfego, dissecação de mensagens e trechos de código funcional do projeto.

Com base no exposto, antes de prosseguir para a discussão acerca dos protocolos, inicialmente, apresenta-se uma breve descrição sobre o projeto que foi desenvolvido e que servirá de base para a análise prática dos protocolos.

Com intuito didático e com base no objetivo deste trabalho, após sua elaboração final, o escrito será disponibilizado publicamente em post realizado a partir do github.

Link do repositório da publicação:

https://github.com/ssamm21/Estudo_aplicado_Protocolos_de_Comunicao

1. Apresentação do sistema - BPM Frotas

O sistema BPM Frotas é uma aplicação web projetada especificamente para a gestão e o controle centralizado de frotas de veículos empresariais. Para a execução do projeto, adotou-se a linguagem de programação JavaScript. Na camada de servidor *backend*, utilizou-se o ambiente de execução Node.js, garantindo um processamento assíncrono para lidar com múltiplas requisições simultâneas. Para a interface com o usuário *frontend*, empregou-se a biblioteca React, proporcionando uma navegação mais fluida. Como banco de dados da aplicação, foi utilizado sistema gerenciador relacional MariaDB, sendo todo o ambiente local configurado e executado sob o sistema operacional Windows.



Logo da aplicação - BPM Frotas

2. TCP/IP

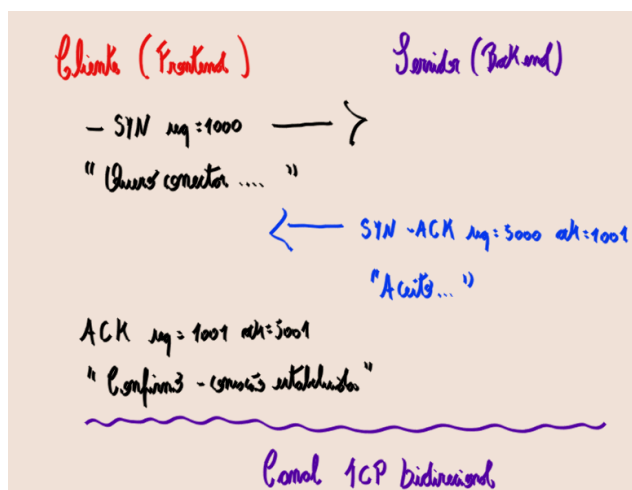
Conforme fora visto, o Sistema de Gestão de Frotas é uma API REST que gerencia veículos, motoristas, alocações, manutenções e despesas. Sua arquitetura distribui a comunicação entre três camadas, cada uma usando protocolos distintos. Todos esses protocolos operam sobre a pilha TCP/IP, que fornece a base confiável de transporte. A tabela abaixo situa cada protocolo no modelo de referência:

Camada TCP/IP	Protocolos do Projeto
Aplicação	HTTP, JWT, SMTP
Transporte	TCP
Internet	IP
Acesso à rede	Ethernet / WiFi

Tabela 1 - Protocolos

Com base nisso, ao abordar o protocolo TCP/IP, identifica-se nele a base para todo o tráfego. Com base nisso, o IP (Internet Protocol) é responsável por endereçar e rotear pacotes entre origens e destinos em redes diferentes. Porém, ele não garante entrega, nem ordem, nem integridade, isto é, pacotes podem ser perdidos, duplicados ou chegar fora de sequência. O TCP (Transmission Control Protocol, RFC 793) adiciona confiabilidade sobre o IP por meio de numeração de sequência, confirmação (ACK), controle de fluxo e checksum, o qual detecta corrupção de dados em trânsito.

Antes de qualquer byte de aplicação trafegar, o TCP realiza uma negociação em três passos para sincronizar os números de sequência de ambos os lados. Tais passos são de importância, uma vez que é por meio deles que ambos os lados têm certeza de que a conexão está ativa e os números de sequência estão sincronizados.



Abrir e fechar uma conexão TCP a cada query SQL seria custoso, necessitaria dois handshakes, abertura e encerramento, por operação. O projeto busca resolver isso com um conjunto de conexões TCP pré-estabelecidas e que são reutilizáveis:

```
JS db.js > ...
1  const mariadb = require('mariadb');
2
3  const pool = mariadb.createPool({
4    host: process.env.DB_HOST,
5    user: process.env.DB_USER,
6    password: process.env.DB_PASSWORD,
7    database: process.env.DB_NAME,
8    connectionLimit: parseInt(process.env.DB_LIMIT) || 5
9  });
10
```

O padrão usado em todas as rotas do projeto:

```
conn = await pool.getConnection(); // pega conexão disponível (TCP já estabelecido)
const rows = await conn.query(
  `SELECT v.id, v.placa, v.cor, v.tipo_veiculo, v.qtd_eixos, v.capacidade_carga, v.status, v.obs,
    v.km_atual, v.km_proxima_revisao, v.data_proxima_revisao, v.data_licenciamento, v.data_seguro,
    v.modelo_id, mo.nome AS modelo_nome, mo.ano,
    ma.id AS marca_id, ma.nome AS marca_nome
  FROM veiculos v
  JOIN modelos mo ON mo.id = v.modelo_id
  JOIN marcas ma ON ma.id = mo.marca_id
  ORDER BY v.placa`
);
res.json(rows);
} catch (err) {
  res.status(500).json({ error: "Erro ao listar veículos: " + err.message });
} finally {
  if (conn) conn.end(); // devolve ao pool - a conexão TCP permanece aberta para a próxima requisição
}
```

O HTTP também aproveita o TCP persistente entre frontend e backend. Os cabeçalhos keep-alive e Connection: Keep-Alive: timeout=5 nas respostas instruem o cliente a não fechar a conexão TCP após cada requisição, reutilizando-a para as próximas pelo período configurado. Em uma SPA que dispara várias requisições ao carregar uma tela, isso elimina múltiplos handshakes em sequência

3. HTTP/1.1 e Arquitetura REST

O HTTP, *Hypertext Transfer Protocol*, é o protocolo de aplicação que define como clientes e servidores trocam mensagens. É stateless, desse modo, cada requisição é independente, o servidor não guarda memória de interações anteriores. Esse design simplifica a escalabilidade, qualquer instância do servidor pode atender qualquer requisição, mas exige que o cliente reenvie sua identidade a cada chamada, papel exercido pelo JWT. Uma mensagem HTTP tem estrutura fixa:

REQUISIÇÃO	RESPOSTA
POST /api/usuarios/login HTTP/1.1 Host: localhost:3001 Content-Type: application/json {"email":"...", "senha":"..."}	HTTP/1.1 200 OK Content-Type: application/json Content-Length: 399 Connection: keep-alive {"mensagem":"Login realizado..."}

Tabela 2 - Modelo Requisição-Resposta

O REST (Representational State Transfer) é um estilo arquitetural que define como usar os métodos HTTP de forma uniforme e previsível. O projeto aplica todos os verbos REST:

Método	Rota no projeto	Código de Sucesso
GET	GET /api/veiculos	200 OK
POST	POST /api/veiculos	201 Created
PUT	PUT /api/veiculos/:id	200 OK
PATCH	PATCH/api/veiculos/:id/indisponivel	200 OK
DELETE	DELETE /api/veiculos/:id	200 OK

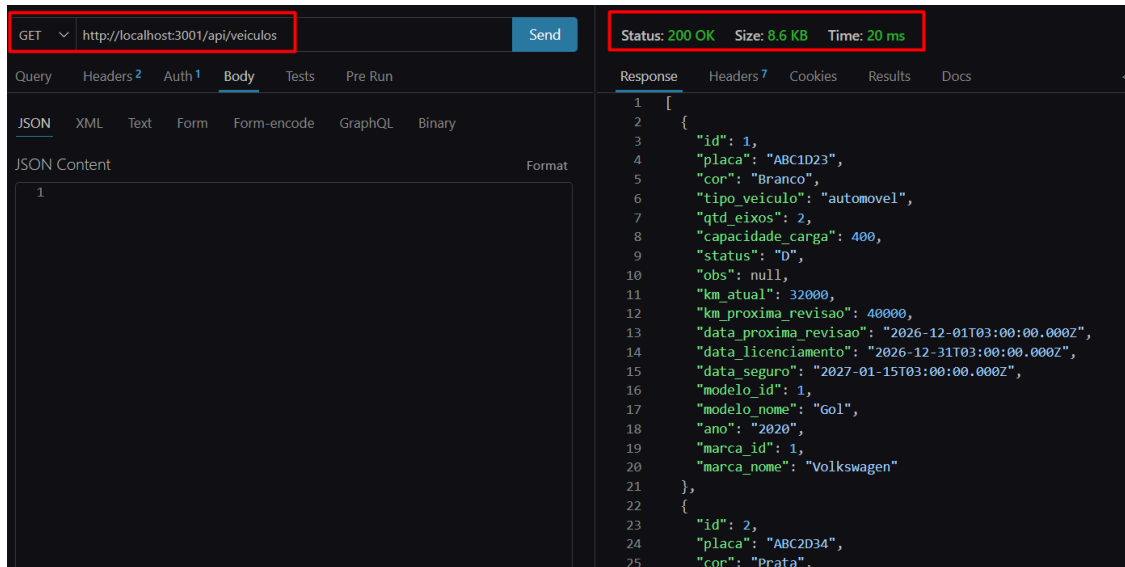
Tabela 3 - Exemplificação dos métodos

O uso de PATCH para mudança de status é semanticamente correto: altera apenas o campo status, sem exigir reenvio de todos os dados:

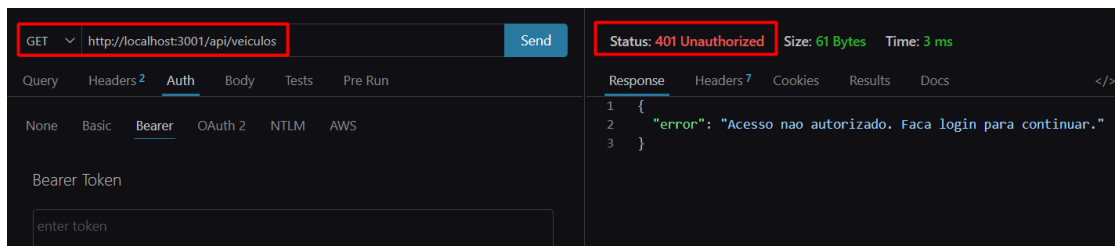
```
// PATCH /:id/indisponivel - marcar veículo como Indisponível (somente se estiver Disponível)
router.patch('/:id/indisponivel', async (req, res) => {
  let conn;
  try {
    conn = await pool.getConnection();
    const rows = await conn.query("SELECT status FROM veiculos WHERE id = ?", [req.params.id]);
    if (rows.length === 0) return res.status(404).json({ error: "Veículo não encontrado." });
    if (rows[0].status !== 'D') {
      return res.status(400).json({ error: "Apenas veículos disponíveis podem ser marcados como indisponíveis." });
    }
    await conn.query("UPDATE veiculos SET status = 'I' WHERE id = ?", [req.params.id]);
    res.json({ message: "Veículo marcado como indisponível." });
  } catch (err) {
    res.status(500).json({ error: "Erro ao atualizar status: " + err.message });
  } finally {
    if (conn) conn.end();
  }
});
```

Exemplos realizados através de thunder client:

- Listar veículos:



- Listar veículos sem token (401 Unauthorized)



4. JWT: Autenticação Stateless

Como o HTTP é stateless, o servidor não sabe quem está fazendo cada requisição. A solução, sessões no servidor, passa a exigir armazenamento centralizado e cria um ponto único de falha em sistemas distribuídos. O JWT, *JSON Web Token*, outro dos protocolos utilizados, resolve isso da seguinte forma: toda a informação de identidade fica dentro do próprio token, assinado criptograficamente. Qualquer instância do backend pode verificar o token sem consultar banco de sessões, a validade, desse modo, é provada pela assinatura.

Analisando a estrutura, um JWT é três blocos **Base64url** separados por pontos. A seguir, o token real gerado pelo sistema durante a sessão de teste é dissecado:

```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJpZiI6Im51bm9tZSI6IjE2ZG1pbnlzdHJhZG9yIiwiaWF0IjoxNjY3MzQ5LCJleHAiOjE3
ODI2OTYxND19
.MMBostN1dsI05yCzZbD6Dq3KK0JoqaSUcBkd3cELhXQ

```

Bloco 1 - Header (algoritmo e tipo):

```
{"alg": "HS256", "typ": "JWT" }
```

Bloco 2 — Payload (dados do usuário — as claims). Exemplo:

```
{  
  "id": 1,  
  "nome": "Administrador",  
  "email": "admin@frotas.com",  
  "tipo_usuario": "Administrador",  
  "iat": 1782667349, ← issued at — 28 Jun 2026 17:22:29 UTC  
  "exp": 1782696149 ← expiration — 29 Jun 2026 01:22:29 UTC (+8h)  
}
```

Bloco 3 — Signature (prova de autenticidade):**HMAC-SHA256**

```
( Base64url(header) + "." + Base64url(payload),  
  segredo = "frotas_jwt_secret_2026"
```

```
)
```

```
→ MMBostN1dsIO5yCzZbD6Dq3KK0JoqaSUcBkD3cELhXQ
```

O HMAC-SHA256 produz um código de autenticação de mensagem usando SHA-256. Se qualquer bit do header ou payload for alterado, por exemplo, mudar "tipo_usuario": "Operador" para "Administrador", a assinatura calculada não baterá com a original, e o servidor rejeitará o token imediatamente.

Geração (no login):

```
const token = jwt.sign(  
  {  
    id:          usuarioDoBanco.id,  
    nome:        usuarioDoBanco.nome,  
    email:       usuarioDoBanco.email,  
    tipo_usuario: usuarioDoBanco.tipo_usuario,  
  },  
  JWT_SECRET,  
  { expiresIn: '8h' }  
);  
res.status(200).json({ mensagem: 'Login realizado com sucesso!', token, usuario:
```

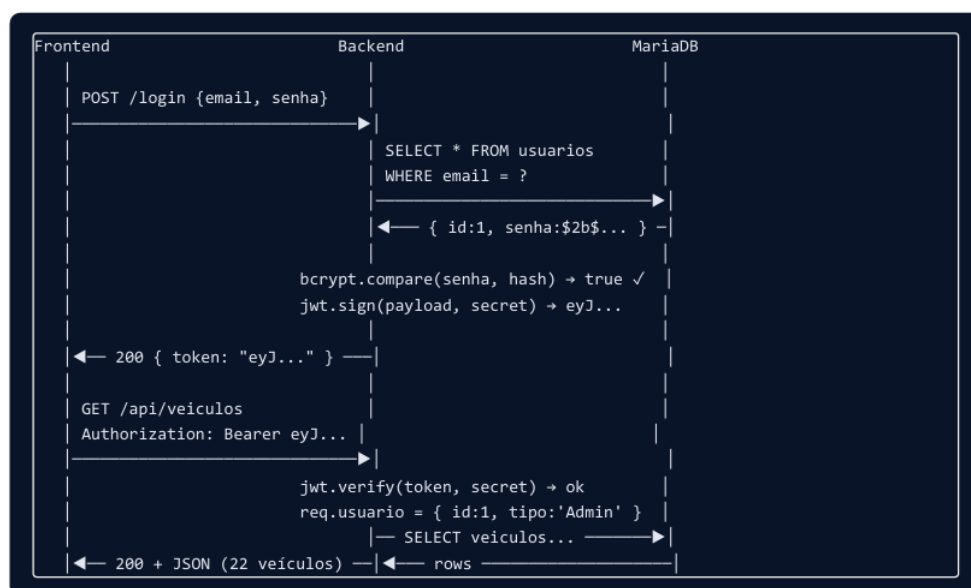
Validação (em toda requisição protegida):

```

57 const ROTAS_PUBLICAS = [
58   'POST /api/usuarios/login',
59   'POST /api/usuarios/esqueceu-senha',
60   'POST /api/usuarios/resetar-senha',
61 ];
62
63 function auth(req) {
64   const rota = `${req.method} ${req.path}`;
65
66   if (ROTAS_PUBLICAS.includes(rota)) {
67     return { ok: true, motivo: 'rota pública - sem token necessário' };
68   }
69
70   const authHeader = req.headers['authorization'];
71
72   if (!authHeader || !authHeader.startsWith('Bearer ')) {
73     return { ok: false, status: 401, error: 'Acesso nao autorizado. Faca login para continuar.' };
74   }
75
76   try {
77     const tokenRecebido = authHeader.split(' ')[1];
78     const usuario = jwt.verify(tokenRecebido, JWT_SECRET);
79     // jwt.verify lança exceção se: assinatura inválida, token expirado,
80     // token malformado ou segredo diferente
81     return { ok: true, usuario };
82   } catch (err) {
83     return { ok: false, status: 401, error: 'Token invalido ou expirado. Faca login novamente.' };
84   }
85 }

```

Esboço do fluxo completo com autenticação:



As senhas são armazenadas como hash `bcrypt`, nunca em texto claro. O `bcrypt` aplica o algoritmo Blowfish com um fator de custo que controla o número de iterações, quanto maior, mais lento e mais resistente a força bruta:

```
bcrypt.hash("admin123", custo=10)
↓
gera salt aleatório de 128 bits
↓
executa 210 = 1024 rounds do algoritmo Blowfish
↓
"$2b$10$eImiTXuWVxfM37uY4JANjQ.7pChkxkJGk6J0RyPkDG0TgvHnNm5K2"
```

ver	cost	salt (22 chars)	hash (31 chars)
-----	------	-----------------	-----------------

O *salt*, uma sequência aleatória de dados gerada no momento da criação da senha, embutido no *hash* garante que dois usuários com a mesma senha tenham resultados finais completamente diferentes, tornando o uso de *rainbow tables* inútil. Como uma *rainbow table* consiste em um vasto banco de dados contendo senhas comuns e seus respectivos *hashes* pré-calculados, a introdução de um *salt* exclusivo para cada registro obriga um possível atacante a recalcular toda a tabela do zero para cada credencial vazada.

Dessa forma, ao aliar essa imprevisibilidade ao alto custo de processamento exigido pelos múltiplos ciclos do algoritmo Blowfish no *bcrypt*, a arquitetura do sistema inviabiliza ataques de dicionário ou força bruta em larga escala, assegurando a proteção das identidades gerenciadas na base de dados.

Conclusão

Por meio do presente estudo, com base no exposto, foi possível compreender e demonstrar a aplicabilidade prática dos protocolos TCP/IP, HTTP/REST e JWT no desenvolvimento do sistema de gestão BPM Frotas. A análise evidenciou que a fundamentação teórica de redes de computadores é indispensável para a construção de arquiteturas de software que sejam, ao mesmo tempo, eficientes e seguras.

A compreensão detalhada da pilha TCP/IP, por exemplo, permitiu a otimização da comunicação com o banco de dados MariaDB por meio do reúso de conexões (uso de *pool*), reduzindo a latência e o custo de processamento. Em paralelo, a adoção do padrão HTTP/1.1 sob a arquitetura REST garantiu uma interface de comunicação uniforme, semântica e escalável entre o cliente (frontend) e o servidor (backend), utilizando corretamente os verbos HTTP para cada operação.

A implementação do protocolo JWT, por fim, aliada a técnicas de criptografia como o algoritmo *bcrypt* com *salt*, provou-se uma solução robusta para o desafio de manter uma

autenticação *stateless*. Essa abordagem não apenas facilitou a escalabilidade do servidor, mas também assegurou a proteção das identidades dos usuários contra ataques cibernéticos, como força bruta e *rainbow tables*.

Dessa forma, para concluir, o escrito buscou destinar um olhar um pouco aprofundado ao projeto desenvolvido, no entanto, neste momento, sob à luz dos conteúdos que vinham sendo trabalhados ao longo da disciplina de Redes e Comunicações. Assim, para concluir, a pesquisa proporcionou um passo a mais, não só entre a integração das disciplinas, como também um bom fundamento para os próximos projetos que serão desenvolvidos no decorrer dos passos do Curso de Ciência da Computação.

Referências

RFC 793. **Transmission Control Protocol**. Postel, J. IETF, 1981. [RFC 793: Transmission Control Protocol | RFC Editor](#)

RFC 7230. **HTTP/1.1: Message Syntax and Routing**. Fielding, R.; Reschke, J. IETF, 2014.

RFC 7519. **JSON Web Token (JWT)**. Jones, M.; Bradley, J.; Sakimura, N. IETF, 2015.
<https://www.rfc-editor.org/rfc/rfc7519>

TANENBAUM, A. S.; WETHERALL, D. **Redes de Computadores**. 5. ed. Pearson, 2011.